

ShoppYGlobe Backend API

API Testing Documentation

Name: Maneesh Kumar

Batch: Jan2026

1. GET /products

The screenshot shows an API testing tool interface. At the top, there are tabs for 'server.js', 'New Request', 'Product.js', '.env', and 'package.json'. Below the tabs, the request method is set to 'GET' and the URL is 'http://localhost:3000/products'. A 'Send' button is visible on the right. Below the request bar, there are tabs for 'Query', 'Headers 2', 'Auth', 'Body 1', 'Tests', and 'Pre Run'. The 'Query' tab is selected. Below the tabs, the response status is '200 OK', the size is '494 Bytes', and the time is '39 ms'. The response body is a JSON array of three product objects. A 'Copy' button is visible on the right side of the response body. At the bottom right, there are buttons for 'Response' and 'Chart'.

```
1  [  
2    {  
3      "_id": "6a7db8d9f909510e3f7382cd",  
4      "name": "iPhone 15",  
5      "price": 69999,  
6      "description": "Apple smartphone",  
7      "stock": 10,  
8      "__v": 0  
9    },  
10   {  
11     "_id": "6a7db923f909510e3f7382ce",  
12     "name": "Sony Headphones",  
13     "price": 4999,  
14     "description": "Wireless headphones",  
15     "stock": 20,  
16     "__v": 0  
17   },  
18   {  
19     "_id": "6a7db938f909510e3f7382cf",  
20     "name": "Nike Shoes",  
21     "price": 5999,
```

2. MongoDB Products Collection

ORGANIZATION
Homi.J's Org - 2026-...

PROJECT
Project 0

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

- admin
- local
- sample_mflix
- shoppyglobe
 - products
- test

products

Cluster0 > shoppyglobe > products

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT CODE

25 1 - 4 of 4

View monitoring

Visualize Your Data

```
_id: ObjectId('6a7dbbaf4470a6129dc1a38f')
name: "iPhone 15"
price: 69999
description: "Apple smartphone"
stock: 10
__v: 0
```

```
_id: ObjectId('6a7dbbbd4470a6129dc1a390')
name: "Sony Headphones"
price: 4999
description: "Wireless headphones"
stock: 20
__v: 0
```

```
_id: ObjectId('6a7dbbca4470a6129dc1a391')
```

3. GET /products/:id

The screenshot shows a VS Code editor with a REST client interface. The top bar has tabs for `server.js`, `TC New Request`, `TC New Request`, `JS Product.js`, `.env`, and `TC New Request`. The REST client interface shows a GET request to `http://localhost:3000/products/6a7dbbaf4470a6129dc1a38f` with a `Send` button. Below the request bar are tabs for `Query`, `Headers`, `Auth`, `Body`, `Tests`, and `Pre Run`. The `Query` tab is active, showing a table for query parameters with columns `parameter` and `value`. Below the query parameters, the response status is `200 OK`, size is `119 Bytes`, and time is `42 ms`. The response body is a JSON object:

```
{
  "_id": "6a7dbbaf4470a6129dc1a38f",
  "name": "iPhone 15",
  "price": 69999,
  "description": "Apple smartphone",
  "stock": 10,
  "__v": 0
}
```

 At the bottom, the `Terminal` tab is active, showing the command `node server.js` and its output:

```
PS C:\Users\royal\Downloads\Kapil\backend> node server.js
Server running on http://localhost:3000
MongoDB connected
█
```

server.js TC New Request TC New Request JS Product.js .env TC New Request

GET http://localhost:3000/products/6a7dbbaf4470a6129dc1a38f Send

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

parameter	value
-----------	-------

Status: 200 OK Size: 119 Bytes Time: 42 ms Response

```
1 {
2   "_id": "6a7dbbaf4470a6129dc1a38f",
3   "name": "iPhone 15",
4   "price": 69999,
5   "description": "Apple smartphone",
6   "stock": 10,
7   "__v": 0
}
```

Response Chart

Problems Output Terminal ...

PS C:\Users\royal\Downloads\Kapil\backend> node server.js
Server running on http://localhost:3000
MongoDB connected
█

4. POST /register

The screenshot shows the VS Code REST Client interface. At the top, there are tabs for 'server.js', 'New Request', 'User.js', 'Product.js', '.env', and 'package.json'. The 'New Request' tab is active. Below the tabs, the request method is set to 'POST' and the URL is 'http://localhost:3000/register'. A 'Send' button is on the right. The 'Body' tab is selected, showing a JSON payload:

```
{  "username": "shoppyuser",  "password": "password123"}
```

. Below the body, the status is '201 Created', size is '42 Bytes', and time is '254 ms'. The 'Response' tab is selected, showing a JSON response:

```
{  "message": "User registered successfully"}
```

. At the bottom, there are tabs for 'Problems', 'Output', and 'Terminal'.

JS server.js TC New Request JS User.js JS Product.js .env package.json

POST http://localhost:3000/register Send

Query Headers² Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "username": "shoppyuser",
3   "password": "password123"
4 }
```

Status: 201 Created Size: 42 Bytes Time: 254 ms Response

```
1 {
2   "message": "User registered successfully"
3 }
```

Response Chart

5. POST /cart

POST ⌵ http://localhost:3000/cart Send

Query

Headers ³

Auth

Body ¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

```
1  {
2    "productId": "6a7dbbaf4470a6129dc1a38f",
3    "quantity": 2
4  }
```

Status: 201 Created

Size: 130 Bytes

Time: 155 ms

Response ⌵

```
1  {
2    "userId": "6a7dbf78ca7731678ae369c2",
3    "productId": "6a7dbbaf4470a6129dc1a38f",
4    "quantity": 2,
5    "_id": "6a7dc63436c697ba16fea3ca",
6    "__v": 0
7  }
```

Copy

Response

Chart

↕

Problems

Output

Terminal

...

node ⚠ + ⌵ 📄 🗑 ⋮ | 🔍 ✕

```
PS C:\Users\royal\Downloads\Kapil\backend> node server.js
Server running on http://localhost:3000
MongoDB connected
█
```

6. PUT /cart/:id

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/cart/6a7dc63436c697ba16fea3ca`. The request body is a JSON object with `productId` and `quantity` fields. The response is a 200 OK status with a JSON body containing `_id`, `userId`, `productId`, `quantity`, and `__v` fields.

Request:

```
PUT http://localhost:3000/cart/6a7dc63436c697ba16fea3ca
```

Body (JSON):

```
{
  "productId": "6a7dbbaf4470a6129dc1a38f",
  "quantity": 2
}
```

Status: 200 OK **Size:** 130 Bytes **Time:** 57 ms

Response (JSON):

```
{
  "_id": "6a7dc63436c697ba16fea3ca",
  "userId": "6a7dbf78ca7731678ae369c2",
  "productId": "6a7dbbaf4470a6129dc1a38f",
  "quantity": 2,
  "__v": 0
}
```

Terminal:

```
PS C:\Users\royal\Downloads\Kapil\backend> node server.js
)` and `findOneAndReplace()` is deprecated. Use `returnDocument: 'after'` instead
```

7. DELETE /cart/:id

DELETE

▼

http://localhost:3000/cart/6a7dc63436c697ba16fea3ca

Send

Query

Headers³

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

```
1  {
2    "productId": "6a7dbbaf4470a6129dc1a38f",
3    "quantity": 2
4  }
```

Status: 200 OK

Size: 39 Bytes

Time: 57 ms

Response ▼

```
1  {
2    "message": "Product removed from cart"
3  }
```

Response

Chart ↗

Problems

Output

Terminal

...

node ⚠

+

▼

📄

🗑

...

|

🔍

✕

8. MongoDB Cart Collection

ORGANIZATION
Homi.J's Org - 2026...

PROJECT
Project 0

Ask AI

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

- admin
- local
- sample_mflix
- shoppyglobe
 - carts**
 - products
 - users
- test

carts

Cluster0 > shoppyglobe > carts

Documents 0AggregationsSchemaIndexes 1Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATABULKEXPORT CODE

250 - 0 of 0

View monitoring

Visualize Your Data

Explain

Reset

Find

Options

This collection has no data

It only takes a few seconds to import data from a JSON or CSV file.

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

9. Unauthorized Cart Access

POST ☐ http://localhost:3000/cart Send

Query Headers² Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {  
2   "productId": "6a7dbbaf4470a6129dc1a38f",  
3   "quantity": 1  
4 }
```

Status: **401 Unauthorized** Size: **42 Bytes** Time: **219 ms** Response ☐

```
1 {  
2   "message": "Access denied. Please login."  
3 }
```

Copy

Response Chart